Comparison and Bounds

19 September 2025 11:29

Comparable<T>

The interface Comparable<T> declares a single instance method for comparing one object with another:

```
interface Comparable<T>{
    public int compareTo(T o);
}
```

The compare to method returns an integer value that is negative, zero or positive. Depending upon whether the receiver-this Object - is less than equal to or greater than the argument. When a class implements Comparable, the Ordering specified by this interface is called the natural ordering for that class.

Typically, an object belonging to a class can only be compared with an object belonging to the same class. For instance, Integer implements Comparable<Integer>:

```
Integer int0 = 0
Error: ';' expected
Integer int0 = 0;
Integer int1 = 1;
int x = int0.compareTo(int1);
x
-1   (int)
int0 = 82
<object reference>   (Integer)
int1 = 72
<object reference>   (Integer)
x = int0.compareTo(int1);
x
1   (int)
x = int0.compareTo(82);
x
0   (int)
```

The comparison returns are negative number since zero precedes one and under numerical ordering. Similarly, String implements Comparable <String>:

```
String str0 = "Zero";
String str1 = "One";
x = str0.compareTo(str1);
x
11   (int)
x = str1.compareTo(str0);
x
-11   (int)
```

Integer

String

Integer & String can't be compared.

```
x = int0.compareTo(str0);
Error: incompatible types: java.lang.String cannot be converted to java.lang.Integer
```

Comparison is not supported between arbitrary numerical types:

```
Number m = Integer.valueOf(2);
Number n = Double.valueOf(3.14);
int x = m.compareTo(n);
```

Undeclared method: compareTo(java.lang.Number)

Here the comparison is illegal because the Number class does not implement the Comparable interface.

Comparison differs from equality in that it does not accept a null argument. If X is not null, X .equals (null) must return false, while X.compareTo(null) must throw a NullPointerException.

We adapt standard idioms for comparison, writing x.compareTo(y) < 0 instead of x < y, and writing x.compareTo(y) <= 0 instead of x <= y.

Contract for Comparable: The contract for the Comparable<T> Interface specifies 3 properties. The properties are defined using the sine function. Which is defined such that sgn(x) returns -1, 0 or 1, Depending on whether 'x' is -ve, 0 or +ve.

First comparison is anti-symmetric. Reversing the order of argument reverses the result.

sgn(x.compareTo(y)) == -sgn(y.compareTo(x))

This generalizes the property for numbers: x < y if and only if y > x. It is also required that x.compareTo(y) raises an exception if and only if y.compareTo(x) raises an exception.

Taking x and y to be the same gives us sgn(x.compareTo(x)) == -sgn(x.compareTo(x)). It follows that:

x.compareTo(x) == 0

So comparison is reflexive -- that is, every value compares as the same as itself.

Second, comparison is transitive. If one value is smaller than a second, and second is smaller than third, then first is smaller than third:

if x.compareTo(y) < 0 and y.compareTo(z) < 0 then x.compareTo(z) < 0.

This generalizes the property for numbers: if x < y and y < z then x < z.

Third, Comparison is a congruence. If two values compare as the same, then they compare the same way with the 3rd value:
if x.compareTo(y) == 0 then sgn(x.compareTo(z)) == sgn(y.compareTo(z))

This generalizes the property for numbers: if x == y then x < z if and only if y < z. Presumably, it is also required that if x.compareTo(y) == 0 then x.compareTo(z) raises an exception if and only if y.compareTo(z) raises an exception, although this is not explicitly stated.

Consistent with Equals: The contract for Comparable strongly recommends that the compareTo() method should be consistent with equals--that is that two objects should satisfy the equals() method if and only if they compare as the same:

x.equals(y) if and only if x.compareTo(y) == 0

Notice that this is a recommendation not a mandatory part of the contract but in fact it is the normal case most classes having a natural ordering do comply with this recommendation If you are designing a class that implements Comparable you should not ignore it without good reason The best example in the platform library that contravenes this recommendation is Java.math.BigDecimal Two instances of this class representing the same value but with different

precision For example 4.0 and 4.00 will compare as the same but not satisfy the equals method One consequence of this design choice is that values of such class cannot be reliably stored in an internally ordered collection like NavigableSet or NavigableMap

Home work:
Wap to show the demo of BigDecimal class.

Maximum of a Collection
How to use the Comparable interface to find the maximum element in a Collection.
public static <T extends Object & Comparable<? super T>> T max(Collection<? extends T> coll)

```
package GenericsAndCollections;

/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student implements Comparable<Student>{
   private int rollNum;
   private String name;
   private String email;

   public Student(int rollNum, String name, String email) {
      this.rollNum = rollNum;
      this.name = name;
      this.email = email;
   }

   @Override
   public int compareTo(Student o) {
      // TODO Auto-generated method stub
      return Integer.compare(this.rollNum, o.rollNum);
   }

   public int getRollNum() {
      return rollNum;
   }

   public String getName() {
      return name;
   }

   public String getEmail() {
      return email;
   }

   public void setRollNum(int rollNum) {
      this.rollNum = rollNum;
   }

   public void setName(String name) {
```

```
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    @Override
    public String toString() {
        return "Student [rollNum=" + rollNum + ", name=" + name + ", email=" + email + "]";
    }

}

package GenericsAndCollections;
import java.util.*;
import java.math.*;

/**
 * Write a description of class Test here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Test
{
    public static void main(String args[]){

        List<Double> nums = new ArrayList<>();
        nums.add(2.0);
        nums.add(3.5);
        nums.add(0.5);
        nums.add(0.25);
        nums.add(-1.35);
        System.out.println(nums);

        Number m = max(nums);
        System.out.print("\nMaximum value: " + m);

        Scanner sc = new Scanner(System.in);
        List<Student> stu = new ArrayList<>();
        for(int i = 0; i < 5; i++){
            System.out.print("\nEnter roll number, name and email: ");
            int rn = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String email = sc.nextLine();
            Student st = new Student(rn, name, email);
            stu.add(st);
        }

        //String maxEmail = max(stu);
        Student s = max(stu);
```

```
        System.out.print("\nMaximum roll-number: " + s.getRollNum());
    }

    public static <T extends Object & Comparable<? super T>>
    T max(Collection<? extends T> coll){
        Iterator<? extends T> i = coll.iterator();
        T maxValue = i.next();
        while(i.hasNext()){
            T next = i.next();
            if(next.compareTo(maxValue) > 0){
                maxValue = next;
            }
        }
        return maxValue;
    }
}
```

The Comparable<T> interface give fine control over what can and cannot be compared. Say we have a Fruit class with subclasses Apple and Orange. Depending on how we set things up. We may prohibit comparison of apples with oranges Or we may permit such comparison.

In this example prohibits comparison of apples with oranges. Here are the three classes it declares:

```
class Fruit{…}
class Apple extends Fruit implements Comparable<Apple>{…}
class Orange extends Fruit implements Comparable<Orange>{…}
```

Each fruit has a name and a size and two fruits are equal if they have same name and same size. Since we have overridden equals, we have also overridden hashCode to ensure that equal objects have same hash code. Apples are compared by comparing their sizes, and so are oranges. Since Apple implements Comparable <apple > It is clear that you can compare Apple with apples, but not with oranges. The test code builds three lists, one of apple 's, one of oranges and one containing mixed fruits. We may find the maximum of the first two lists, but attempting to find the maximum of the mixed list signals an error at compile time.

The following code example permits comparison of apples with oranges. Compare these three class declaration with those given previously.

```
class Fruit implements Comparable<Fruit>{…}
class Apple extends Fruit {…}
class Orange extends Fruit {…}
```

As before, each fruit has a name and a size, and two fruits are equal if they have the same name and the same size. Now since Fruit implements Comparable <Fruit > any two fruits may be compared by comparing their sizes. So the test code can find the maximum of all three lists, including the one that mixes apple with oranges.

Recall that at the end of the previous section, we extended the type signature of Max to use super:

```
<T extends Comparable<? super T>> T max(Collection<? extends T> coll)
```

It shows why this wild card is needed. If we want to compare two oranges, we take T in the preceding code to be orange.

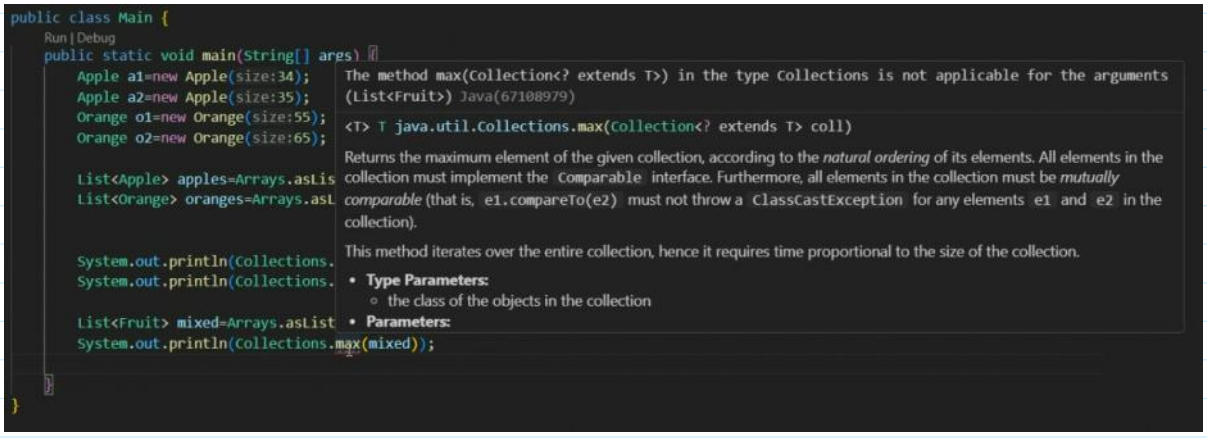
Orange extends Comparable<? super Orange>

And this is true both of the following hold:

Orange extends Comparable<Fruit> and Fruit super Orange

Without the Super wild card, finding the maximum of List <Orange > would be illegal even though finding maximum of a List<Fruit > is permitted.

Also note that natural ordering is used here. Is not consistent with equals. Two fruits with different name but the same size compare as same but they are not equal.




```
public class Fruit implements Comparable<Fruit>{
   String name;
   int size;
   public String getName() {
      return name;
   }
   public void setName(String name) {
      this.name = name;
   }
   public int getSize() {
      return size;
   }
   public void setSize(int size) {
      this.size = size;
   }
   public Fruit(String name, int size) {
      this.name = name;
      this.size = size;
   }

   @Override
   public boolean equals(Object obj){
      if(this==obj){
         return true;
      }
      if(!(obj instanceof Fruit)){
         return false;
      }
      Fruit other=(Fruit)obj;
      return name.equals(other.name) && size==other.size;
```

```
    }

    @Override
    public int hashCode(){
        return name.hashCode()*31+size;
    }

    @Override
    public String toString(){
        return "Name:"+this.name+" , Size:"+this.size;
    }

    @Override
    public int compareTo(Fruit other){
        return Integer.compare(this.size, other.size);
    }
}


public class Apple extends Fruit {
    public Apple(int size){
        super("Apple",size);
    }
}

public class Orange extends Fruit {
      public Orange(int size){
          super("Orange", size);
      }
}


import java.util.Arrays;
import java.util.Collection;
import java.util.Collections;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Apple a1=new Apple(34);
        Apple a2=new Apple(35);
        Orange o1=new Orange(55);
        Orange o2=new Orange(65);

        List<Apple> apples=Arrays.asList(a1,a2);
        List<Orange> oranges=Arrays.asList(o1,o2);


        System.out.println(Collections.max(apples));
        System.out.println(Collections.max(oranges));

        List<Fruit> mixed=Arrays.asList(a1,o1,a2,o2);
         System.out.println(Collections.max(mixed));
```

```
    }
}
```

Comparator<T>

Sometimes we want to compare objects that do not implement the Comparable interface or to compare objects using a different ordering from one specified by that interface. The ordering provided by the Comparable interface is called natural ordering. So the Comparator interface provides, so to speak, an unnatural ordering. We specify additional ordering using the Comparator interface, which declares two abstract methods:

```
interface Comparator<T> {
    public int compare(T o1, T o2);
    public boolean equals(Object obj);
}
```

The compare method returns a value that is negative, 0, or positive depending upon whether the first object is less than, equal to, or greater than the second object just as with compareTo. (The equals method is the one familiar from class Object; it is included in the interface to remind implementers that equal comparators must have compare methods that impose the same ordering.)
//Prohibiting comparison of Apples with Oranges

```
List<Fruit> mixed = List.of(a1, o2, a3, o4);
System.out.print("\nMax: "+Collections.max(mixed)); // compile time error
```

```
no suitable method found for max(java.util.List<GenericsAndCollections.Fruit>)
  method java.util.Collections.<T>max(java.util.Collection<? extends T>) is not applicable
    (inference variable T has incompatible bounds
      upper bounds: java.lang.Object,java.lang.Comparable<? super T>
      lower bounds: GenericsAndCollections.Fruit)
  method java.util.Collections.<T>max(java.util.Collection<? extends T>,java.util.Comparator<? super T>) is not applicable
    (cannot infer type-variable(s) T
      (actual and formal argument lists differ in length))
```

```
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class Fruit here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public abstract class Fruit
{
   protected String name;
   protected int size;

   protected Fruit(String name, int size){
      this.name = name; this.size = size;
   }

   public boolean equals(Object o){
```

```
        if (o instanceof Fruit){
            Fruit that = (Fruit)o;
            return this.name.equals(that.name) &&
            this.size == that.size;
        }
        else{
            return false;
        }
    }

    public int hashCode(){
        return Objects.hash(name, size);
    }

    protected int compareTo(Fruit that){
        return Integer.compare(this.size, that.size);
    }
}

public class Apple extends Fruit implements Comparable<Apple>
{
    public Apple(int size){
        super("Apple", size);
    }
    @Override
    public int compareTo(Apple a){
        return super.compareTo(a);
    }
}

public class Orange extends Fruit implements Comparable<Orange>
{
    public Orange(int size){
        super("Orange", size);
    }
    @Override
    public int compareTo(Orange a){
        return super.compareTo(a);
    }
}

package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class TestAppleAndOrange here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestAppleAndOrange
{
    public static void main(String args[]){
```

```
        Apple a1 = new Apple(1);
        Apple a2 = new Apple(2);
        Apple a3 = new Apple(3);
        Apple a4 = new Apple(4);

        Orange o1 = new Orange(5);
        Orange o2 = new Orange(6);
        Orange o3 = new Orange(7);
        Orange o4 = new Orange(8);

        List<Apple> apples = Arrays.asList(a1, a2, a3, a4);
        System.out.print("\f"+Collections.max(apples).equals(a4)); //true

        List<Orange> oranges = Arrays.asList(o1, o2, o3, o4);
        System.out.print("\n"+Collections.max(oranges).equals(o4)); //true

        List<Fruit> mixed = List.of(a1, o2, a3, o4);
        //System.out.print("\nMax: "+Collections.max(mixed)); //Compile-time error
    }
}
```

Permitting comparison of apples with oranges

```
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class Fruit here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public abstract class Fruit implements Comparable<Fruit>
{
    protected String name;
    protected int size;

    protected Fruit(String name, int size){
        this.name = name; this.size = size;
    }

    public boolean equals(Object o){
        if (o instanceof Fruit){
            Fruit that = (Fruit)o;
            return this.name.equals(that.name) &&
            this.size == that.size;
        }
        else{
            return false;
        }
    }

    public int hashCode(){
```

```
        return Objects.hash(name, size);
    }

    public int compareTo(Fruit that){
        return Integer.compare(this.size, that.size);
    }
    @Override
    public String toString(){
        return "name: " + this.name+ "\nsize: " + this.size;
    }
}

public class Apple extends Fruit
{
    public Apple(int size){
        super("Apple", size);
    }
}


public class Orange extends Fruit
{
    public Orange(int size){
        super("Orange", size);
    }
}
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class TestAppleAndOrange here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestAppleAndOrange
{
    public static void main(String args[]){
        Apple a1 = new Apple(1);
        Apple a2 = new Apple(2);
        Apple a3 = new Apple(3);
        Apple a4 = new Apple(4);

        Orange o1 = new Orange(5);
        Orange o2 = new Orange(6);
        Orange o3 = new Orange(7);
        Orange o4 = new Orange(8);

        List<Apple> apples = Arrays.asList(a1, a2, a3, a4);
        System.out.print("\f"+Collections.max(apples).equals(a4)); //true

        List<Orange> oranges = Arrays.asList(o1, o2, o3, o4);
        System.out.print("\n"+Collections.max(oranges).equals(o4)); //true
```

```
        List<Fruit> mixed = List.of(a1, o2, a3, o4);
        System.out.print("\nMax: "+Collections.max(mixed)); //Ok
    }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options
true
true
Max: name: Orange
size: 8
```

Here is a comparator that considers the shorter of two string to be smaller. Only if two strings have the same length. Are they compared using the natural (alphabetic) ordering.

```
package GenericsAndCollections;
import java.util.*;

public class TestComparator
{
   public static void main(String args[]){
      Comparator<String>  sizeOrder =
         new Comparator<>(){
            public int compare(String s1, String s2){
               return
               s1.length() < s2.length() ? -1 : s1.length() > s2.length() ? 1 : s1.compareTo(s2);
            }
         };

      System.out.print("\f"+ "two".compareTo("three"));
      System.out.print("\n" + sizeOrder.compare("two", "three"));

   }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options
15
-1
```

In the natural alphabetic ordering "two" is greater than "three" whereas in the size ordering it is smaller.

The Java libraries always provide a choice between Comparable and Comparator. For every generic method. With a type variable bounded by Comparable. There is another generic method with an additional argument of type Comparator. For instance corresponding to:

```
public static <T extends Comparable<? super T>> T max(Collection<? extends T> coll)
```

We also have:

```
public static <T> T max(Collection<? extends T coll, Comparator<? super T> comp)
```

There are similar methods to find the minimum. For example, here is how to find the maximum and minimum of a list using the natural ordering and using the size ordering:

```
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class TestComparator here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestComparator
{
   public static void main(String args[]){
      Comparator<String>  sizeOrder =
         new Comparator<>(){
            public int compare(String s1, String s2){
               return
               s1.length() < s2.length() ? -1 : s1.length() > s2.length() ? 1 : s1.compareTo(s2);
            }
         };

      System.out.print("\f"+ "two".compareTo("three"));
      System.out.print("\n" + sizeOrder.compare("two", "three"));

      Collection<String> strings = Arrays.asList("from", "aaa", "to", "zzz");
      System.out.print("\n"+ Collections.max(strings)); //zzz
      System.out.print("\n" + Collections.min(strings));//aaa
      System.out.print("\n" + Collections.max(strings, sizeOrder));//from
      System.out.print("\n" + Collections.min(strings, sizeOrder));//to
   }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options
15
-1
zzz
aaa
from
to
```

The string "from" is the maximum using the size ordering because it is the longest, and "to" is minimum because it is shortest.

Here, slightly simplified, is the code in the class Collection for the version of max that takes a Comparator.

```
public static <T extends Comparable<T>> T max(Collection<T> coll,
   Comparator<? super T> comp){
      Iterator<? extends T> i = coll.iterator();
      T candidate = i.next();
      while(i.hasNext()){
```

Compared to the previous version, the only

```
        Iterator<? extends T> i = coll.iterator();
        T candidate = i.next();
        while(i.hasNext()){
          T next = i.next();
          if(comp.compare(next, candidate) > 0){
            candidate = next;
          }
        }
        return candidate;
      }
```

Compared to the previous version, the only change is that where before we wrote next.compareTo(candicate), now we write compare(next, candicate).

It is easy to define a comparator that provides the natural ordering. This is a slightly simplified version of the code for the static method natural order on the Comparator interface.

```
public static <T extends Comparable<? super T>> Comparator<T> naturalOrder(){
    return new Comparator<>(){
      public int compare(T o1, T o2){
        return o1.compareTo(o2);
      }
    };
  }
```

Using this, we can define the version of Max that uses the natural ordering in terms of the version that uses a given comparator. This is the overload of Max that was used in the string version of collections method in. "Comparable <T>"

```
  public static <T extends Comparable<T>> T max(Collection<T> coll){
    return Collections.max(coll, Comparator.naturalOrder());
  }
```

In everyday practice, you will probably create Comparator instances using one of the static or default methods that the interface exposes, or a combination of them. Let's explore how they work on a simple record type:

```
    record Person(String name, int age){}
```

Probably the most commonly used methods are comparing and its variant, which accepts a function that extracts a keys to use in comparing two objects. For example, if we want to sort a list of Person objects by name, we can use this Comparator.

```
    Comparator<Person> compareByname = Comparator.comparing(Person::name);
```

The method comparing takes a function (represented by functional interface java.util.function.Function -typically a getter-- from the type to be sorted to sort a key. It then returns a comparator that, when it is applied to two objects of the type to be sorted, will return a value corresponding to the natural ordering on their respective sort keys. For example, if we define a list of Person objects:

```
    Person a32 = new Person("Emy", 32);
    Person a23 = new Person("John", 23);
    List<Person> lp = new ArrayList<>(List.of(a32, a23));
```

We can write:

```
    lp.sort(compareByName);
```

Even somewhat simplified the declaration of comparator. Comparing in JDK looks very difficult.

```
public static <T, U extends Comparable<? super U>> Comparator<T>
   comparing(Function<? super T, ? extends U> keyExtractor){
      return (c1, c2)->keyExtractor.apply(c1).compareTo(keyExtractor.apply(c2));
   }
```

In fact, we have met most of the difficulties already. T is the type of the class to be compared and U is the type of sort key. As we saw at the end of. "Maximum of a collection", the most generally useful bond on the type of a comparable is U extends Comparable<? super U>; in other words, if any supertype of U defines compareTo, U will inherit that definition. The type signature of the function interface can be explained by the substitution principle. A function that can be applied to some super type of T can be certainly be applied to T value. Conversely, if it returns a subtype of U, that will be certainly be usable where a U is required.

The intimidating appearance of this declaration contrasts with the ease of using the method that it declares. This contrast is typical of Java generics. The API designer needs to think very carefully about how to make it as usable and general as possible. If the design is successful, client code developers will be highly appreciative. If they stop to think about it, that is, since the best designed Apis are distinguished by their unobtrusiveness.